

```
#!/usr/bin/env python3
```

```
"""
```

```
telegram_signal_bot.py
```

Interactive Telegram bot: send it a chart screenshot, it replies with a BUY_UP / BUY_DOWN / WAIT signal plus stop-loss and position-size numbers, using the same analysis logic as chart_signal_bot.py.

Reminder: this reads a static chart image with a vision model - it is not live market data and it does not know anything a human couldn't see in the same screenshot. Treat every reply as one input, not a guarantee.

Setup:

```
pip install python-telegram-bot anthropic
```

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

```
export TELEGRAM_BOT_TOKEN="..." # from @BotFather
```

Run:

```
python telegram_signal_bot.py
```

Then in Telegram:

/start	- shows current settings
/setaccount 1000	- set account size (USD)
/setrisk 1.5	- set %% risked per trade
/setmaxpos 20	- set max %% of account per trade
Just send a chart photo	- get a signal back

```
"""
```

```
import json
```

```
import logging
```

```
import os
```

```
import sys
```

```
import tempfile
```

```
from telegram import Update
```

```
from telegram.ext import (
```

```
    ApplicationBuilder, CommandHandler, MessageHandler,  
    ContextTypes, filters,
```

```
)
```

```
# reuse the analysis + risk logic from chart_signal_bot.py
```

```
from chart_signal_bot import analyze_chart, build_signal, format_alert
```

```

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

DEFAULTS = {
    "account_size": 1000.0,
    "risk_pct": 1.0,
    "max_position_pct": 20.0,
    "stop_pct": 2.0,
    "min_confidence": 0.55,
}

# per-chat settings, in-memory (resets on restart)
chat_settings: dict[int, dict] = {}

def settings_for(chat_id: int) -> dict:
    return chat_settings.setdefault(chat_id, dict(DEFAULTS))

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    s = settings_for(update.effective_chat.id)
    await update.message.reply_text(
        "Send me a chart screenshot and I'll read it for a BUY_UP / BUY_DOWN / WA  

        signal with stop-loss and position size.\n\n"
        f"Current settings:\n"
        f"  account size: {s['account_size']}\n"
        f"  risk per trade: {s['risk_pct']}%\n"
        f"  max position: {s['max_position_pct']}%\n"
        f"  min confidence: {s['min_confidence']}\n\n"
        "Change with /setaccount, /setrisk, /setmaxpos, /setminconf.\n\n"
        "This is a vision model reading a static image, not live market data - "  

        "treat it as one input, not financial advice."
    )

def _make_setter(key: str, label: str):
    async def handler(update: Update, context: ContextTypes.DEFAULT_TYPE):
        s = settings_for(update.effective_chat.id)
        if not context.args:
            await update.message.reply_text(f"Current {label}: {s[key]}")
            return
        try:
            s[key] = float(context.args[0])
            await update.message.reply_text(f"{label} set to {s[key]}")
        except ValueError:
            await update.message.reply_text("Give me a number, e.g. /setrisk 1.5")
    return handler

```

```

async def handle_photo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    s = settings_for(update.effective_chat.id)
    await update.message.reply_text("Reading chart...")

    photo = update.message.photo[-1] # highest resolution
    tg_file = await photo.get_file()

    with tempfile.NamedTemporaryFile(suffix=".jpg", delete=False) as tmp:
        tmp_path = tmp.name
        await tg_file.download_to_drive(tmp_path)

    try:
        analysis = analyze_chart(tmp_path)
    except SystemExit as e:
        await update.message.reply_text(f"Couldn't read that chart: {e}")
        return
    finally:
        os.unlink(tmp_path)

    if analysis.get("confidence", 0) < s["min_confidence"] and analysis.get("sign
        analysis["signal"] = "WAIT"
        analysis["reasoning"] = f"(forced WAIT: confidence below {s['min_confiden

    result = build_signal(
        analysis,
        account_size=s["account_size"],
        risk_pct=s["risk_pct"],
        max_position_pct=s["max_position_pct"],
        stop_pct=s["stop_pct"],
    )

    text = format_alert(symbol=None, result=result)
    await update.message.reply_markdown(text)

async def handle_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Send me a chart screenshot (as a photo), or

def main():
    token = os.environ.get("TELEGRAM_BOT_TOKEN")
    if not token:
        sys.exit("Set TELEGRAM_BOT_TOKEN in your environment first.")

    app = ApplicationBuilder().token(token).build()

```

```
app.add_handler(CommandHandler("start", start))
app.add_handler(CommandHandler("setaccount", _make_setter("account_size", "ac
app.add_handler(CommandHandler("setrisk", _make_setter("risk_pct", "risk per
app.add_handler(CommandHandler("setmaxpos", _make_setter("max_position_pct",
app.add_handler(CommandHandler("setminconf", _make_setter("min_confidence", "
app.add_handler(MessageHandler(filters.PHOTO, handle_photo))
app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_text))

log.info("Bot starting (polling)...")
app.run_polling()

if __name__ == "__main__":
    main()
```